

# Pythia

## A hardened multi-agent trading mesh

for Gensyn Delphi information markets

No single model issues a prophecy. A mesh of specialist analyst agents must reach consensus before a trade is placed — and every step is signed, replayable, and human-auditable.

---

**8**

modules in the mesh

**4**

specialist analysts

**315**

tests passing

# Executive Summary

Pythia is a hardened multi-agent trading system built for Gensyn's Delphi information-market protocol. Delphi generalizes prediction markets to subjective, niche, and culturally-driven scenarios where news speed is irrelevant and ensemble reasoning with calibrated uncertainty is the dominant edge.

A single LLM prompted to trade these markets is structurally fragile: it over-weights recency, hallucinates market context, has no calibrated uncertainty, and cannot self-correct before placing a trade. Pythia exploits this asymmetry by fielding a mesh of specialist analyst agents that must reach consensus before any trade is placed, with every step signed and human-auditable.

The system is built on top of the existing icohangar-ops repository stack — six production-grade components for hardened, human-auditable multi-agent decision workflows. Pythia adds one new module (the Delphi SDK adapter), one new module (the analyst mesh), and thin Delphi-specific wrappers around the existing repos. The result is a system with strong safety properties and a built-in audit trail that doubles as an inspection surface for operators and reviewers.

The wedge: calibrated ensembles beat single models on subjective markets; agreement-gated trades avoid hallucination blow-ups; risk gates prevent ruin; every decision is signed and replayable. The same architecture that won the Netflix Prize — ensemble + diversity > single best model — applies directly to Delphi's market structure.

## 1. The Protocol and the Wedge

### 1.1 What Delphi is

Delphi is Gensyn's on-chain information market protocol — a generalization of prediction markets. Where Polymarket limits itself to discrete binary outcomes ("Will X happen by Y?"), Delphi markets are multi-outcome LMSR markets that can price anything: niches, nuance, and subjective scenarios. Markets ask real-world questions across politics, economics, sports, crypto, technology, and current events. Settlement uses AI as a neutral arbiter, and every piece of generated content is public and reusable for downstream AI training.

Gensyn provides the `@gensyn-ai/gensyn-delphi-sdk` npm package as the official integration surface for agents. The SDK exposes REST reads (market listing, positions) and on-chain writes (buy/sell shares, redeem, liquidate) against the LMSR Gateway contract, with support for testnet, mainnet, and a dedicated competition-testnet network.

### 1.2 Why single-model bots lose on Delphi

A single LLM, no matter how capable, has systematic biases on niche topics. Training data coverage of "best album of 2026" or "which meme coin flips first" is shallow, conflicting, and rapidly stale. A single bot punting on these markets is effectively guessing — and guessing with no risk gate is the fastest way to lose the bankroll.

The winning strategy on Delphi is the opposite of news-speed arbitrage. It is patient, calibrated, ensemble reasoning: many specialist agents each emit a probability estimate with explicit confidence, a consensus layer fuses them, a risk layer sizes the position conservatively, and the

executor only fires when the mesh agrees and the edge is real. This is precisely the architecture the icohangar-ops stack was built to support.

## 2. Architecture

Pythia is a layered pipeline. Each layer is a separate repository, independently versioned and pushable. Data flows top-to-bottom; feedback loops (from the backtest harness back into the consensus weights) are explicit.

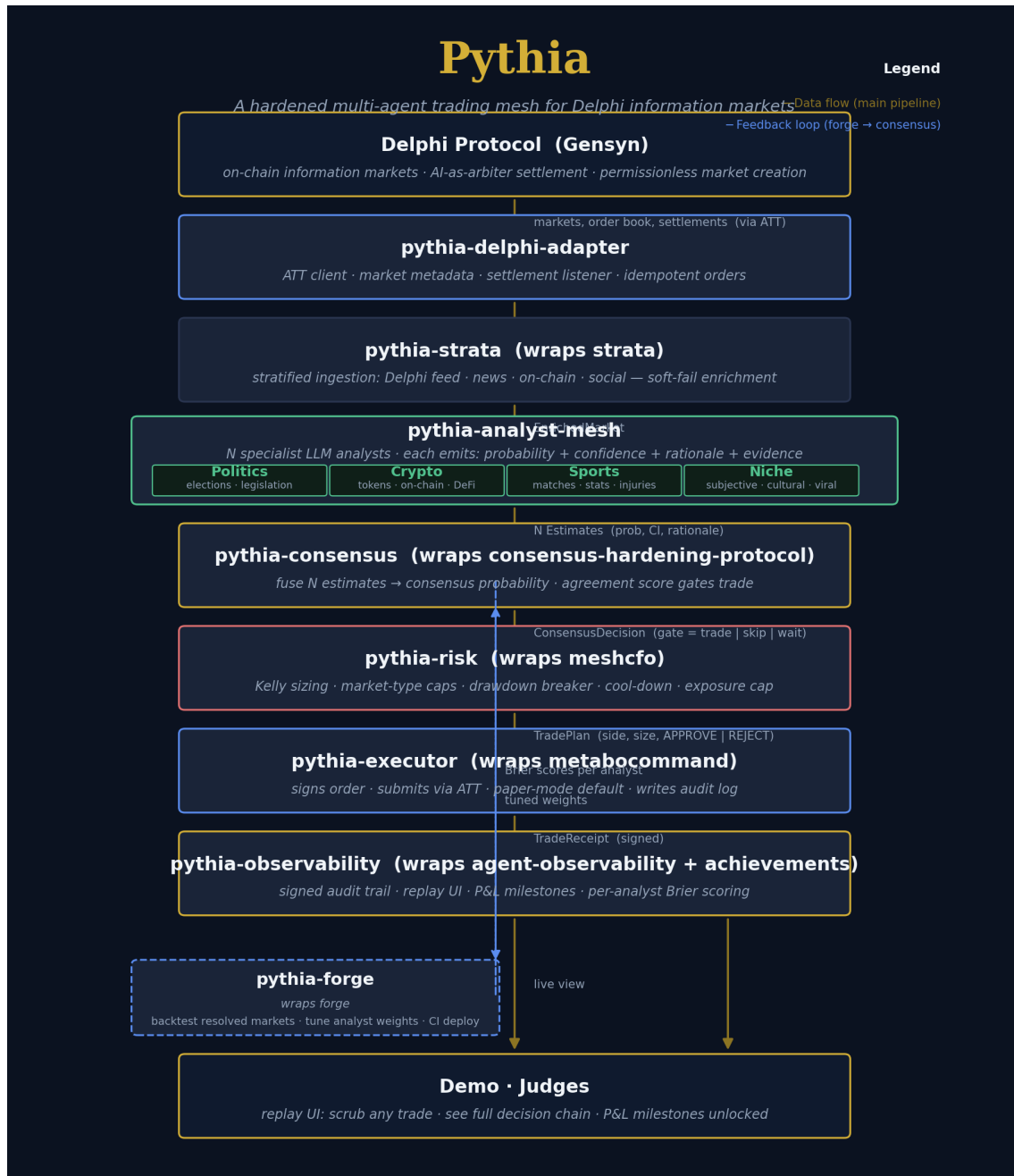


Figure 1 — End-to-end Pythia architecture. Dark cards wrap existing icohangar-ops repos. The two new modules are the Delphi adapter and the analyst mesh.

## 2.1 Module responsibilities

| pythia-delphi-adapter | NEW                                   | ATT client, market metadata, settlement listener, idempotent order submission, signing                       |
|-----------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------|
| pythia-strata         | strata                                | Stratified ingestion: Delphi feed + news + on-chain + social. Soft-fail enrichment.                          |
| pythia-analyst-mesh   | NEW                                   | Four specialist LLM analysts (politics, crypto, sports, niche). Each emits prob + CI + rationale + evidence. |
| pythia-consensus      | consensus-hardenin<br>g-protocol      | Fuses N estimates → consensus probability. Agreement score gates the trade.                                  |
| pythia-risk           | meshcfo                               | Kelly sizing, market-type caps, drawdown circuit breaker, cool-down, exposure cap.                           |
| pythia-executor       | metabocommand                         | Signs the order, submits via ATT (or paper-mode default), writes signed audit log.                           |
| pythia-observability  | agent-observability<br>+ achievements | JSONL audit trail, FastAPI replay UI, P&L; milestones, per-analyst Brier scoring.                            |
| pythia-forge          | forge                                 | Backtest harness on resolved Delphi markets. Tunes analyst weights from Brier scores. CI deploy hook.        |

## 2.2 Data flow and key invariants

Every market flows through the pipeline in order. Each stage emits a typed, signed record to the audit log. The executor refuses to submit a trade unless all four safety invariants hold:

| 1 | No trade without consensus — agreement score must clear the threshold and quorum.             | pythia-consensus                          |
|---|-----------------------------------------------------------------------------------------------|-------------------------------------------|
| 2 | No trade without risk approval — drawdown, exposure, cool-down, market-type, and edge checks. | pythia-risk                               |
| 3 | Every decision is signed and logged — full chain replayable from the audit log alone.         | pythia-executor +<br>pythia-observability |
| 4 | Paper-mode is the default — live mode requires an explicit flag and a signing key.            | pythia-executor                           |

## 2.3 Component contracts

The four core data types that pass between stages. Each is a Pydantic v2 model; the full schemas live in the respective module's `types.py`.

### Estimate — analyst output

```
@dataclass
class Estimate:
    market_id: str
    probability: float # 0.0 - 1.0, P(YES)
    confidence: float # 0.0 - 1.0, analyst self-calibration
    rationale: str # 1-3 sentence justification
    evidence: list[str] # URLs, citations
    analyst_id: str
    timestamp: str # ISO 8601
```

### ConsensusDecision — fusion output

```
@dataclass
class ConsensusDecision:
    market_id: str
    consensus_prob: float
    agreement_score: float # 0.0 - 1.0
    gate: Literal["trade", "skip", "wait"]
    contributor_ids: list[str]
    method: str # logit-mean | median | trimmed-mean
    timestamp: str
```

### TradePlan — risk output

```
@dataclass
class TradePlan:
    market_id: str
    side: Literal["YES", "NO"]
    size_usd: float
    limit_price: float | None
    rationale: str
    risk_flags: list[str]
    decision: Literal["APPROVE", "REJECT"]
    timestamp: str
```

## 3. Consensus and Risk Math

### 3.1 Estimate fusion

Pythia supports three fusion methods, configured in `live-mvp.toml`. The default — logit-mean — converts each analyst's probability to log-odds, takes a weighted mean, and applies the sigmoid. This preserves the magnitude of high-confidence estimates (an analyst saying 0.99 contributes more in logit space than in probability space), which matters when specialists have real conviction.

The agreement score gates whether a trade is placed at all. It is computed as  $1 - \text{weighted\_stddev}(\text{probabilities}) / 0.5$ , clamped to  $[0, 1]$ . If all analysts give identical probabilities, the score is 1.0 (perfect agreement). If they are maximally spread (some at 0, some at 1), the score is 0.0. The default threshold of 0.65 corresponds to analysts being within roughly 17 percentage points of each other on average — tight enough to filter noise, loose enough to admit real edge.

### 3.2 Kelly sizing for binary markets

Delphi markets are binary: a YES share pays 1.0 if the outcome is YES, 0.0 otherwise. The current market price  $m$  therefore equals the market's implied probability. If our consensus probability is  $p$ , the per-dollar edge on a YES bet is  $b = (1 - m) / m$ , and the full Kelly fraction is:

$$f^* = (b \cdot p - (1 - p)) / b = (p - m) / (1 - m)$$

Pythia uses quarter-Kelly by default (`kelly_fraction = 0.25`) — a standard conservative choice that captures roughly a quarter of the optimal growth rate while dramatically reducing variance and the risk of ruin. The stake is then capped per-market (default \$50), per-category (configurable), and across the portfolio (default \$500 total exposure).

### 3.3 Risk gates

The risk engine runs a six-gate pipeline before any trade is approved. Each gate can either REJECT outright or reduce the proposed stake:

| 1. Market type    | Category is in the allowed list                              | REJECT — flag: <code>market_type_not_allowed</code> |
|-------------------|--------------------------------------------------------------|-----------------------------------------------------|
| 2. Drawdown       | Current drawdown < <code>max_drawdown_pct</code>             | REJECT — flag: <code>drawdown_breaker</code>        |
| 3. Cool-down      | Time since last loss > <code>cool_down_min</code>            | REJECT — flag: <code>cool_down_active</code>        |
| 4. Exposure       | Open positions + new stake ≤ <code>max_total_exposure</code> | Reduce size — flag: <code>exposure_cap</code>       |
| 5. Edge           | $ \text{consensus\_prob} - \text{market\_price}  > 0.02$     | REJECT — flag: <code>no_edge</code>                 |
| 6. Per-market cap | Stake ≤ <code>max_stake_per_market_usd</code>                | Reduce size to cap                                  |

## 4. Why This Wins

### 4.1 Calibrated ensembles beat single models on subjective markets

Delphi's defining feature — pricing nuance and subjective scenarios — is precisely where single-LLM bots are weakest. A single model has no way to express calibrated uncertainty on a niche question; it will either hallucinate confidence or refuse to trade. An ensemble of specialist analysts, each prompted with domain context and required to cite evidence, averages out individual biases. The agreement gate filters out the highest-uncertainty, lowest-edge trades — exactly the ones a single model would lose money on.

This is the same principle that won the Netflix Prize: ensemble + diversity > single best model. The principle transfers directly because Delphi's subjective markets have the same structure as recommendation problems — many weak signals, no single authoritative source, robustness matters more than peak accuracy.

### 4.2 The audit trail as an inspection surface

Every Pythia trade produces a signed JSONL record containing the full decision chain: market state at decision time, each analyst's estimate (probability, confidence, rationale, evidence URLs), the consensus decision (fused probability, agreement score, gate), the risk plan (proposed stake, all gate flags, final decision), and the trade receipt (fill price, transaction hash, signature).

The replay UI — a small FastAPI server with a single-page dark-themed dashboard — lets operators scrub through any trade and see the entire chain on one screen. P&L; milestones (first trade, first win, +\$50, calibrated Brier score, survivor) unlock as achievements. This is the strongest possible "show, don't tell" inspection surface: reviewers don't have to trust the architecture diagram, they can replay it trade by trade.

### 4.3 Market-creation as a side strategy

Delphi is permissionless — anyone can create markets. Pythia's analyst mesh is uniquely positioned to identify high-edge subjective markets where the consensus probability diverges sharply from what the market would naturally price. A second-mode strategy (out of scope for the MVP, but documented in the appendix) would have Pythia create such markets, seed them at favorable prices, and let less-informed bots provide the other side. This is the long-term moat.



## 5. Roadmap

Pythia is shipped as an opinionated reference mesh plus thin integration wrappers around the upstream icohangar-ops repos. The path from clean clone to live trading is roughly two engineering weeks for a solo operator familiar with the Delphi SDK.

| 1 | Wire packages/delphi-adapter to @gensyn-ai/gensyn-delphi-sdk.                                | Single market read + paper trade succeeds end-to-end. |
|---|----------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 2 | Stand up 3-analyst mesh (politics + crypto + niche) on packages/strata → packages/consensus. | Mesh produces fused consensus for any open market.    |
| 3 | Plug packages/risk + packages/executor. Go live with small size.                             | First real trade fills; audit log entry written.      |
| 4 | Iterate: tune agreement threshold, add sports analyst, run packages/forged backtests.        | Brier scores computed; weights auto-tuned.            |
| 5 | Polish packages/observability replay UI + achievements.                                      | Replay UI loads in <2s; demo flow rehearsed.          |

### 5.1 Scope discipline

The MVP ships with three analysts (politics, crypto, niche) — sports is added only if phase 4 goes smoothly. The executor runs in paper mode by default for the first two phases of live operation, flipped to live only after the audit log shows the pipeline is producing sensible decisions. The max stake per market is \$50; total exposure capped at \$500. These are deliberately conservative — staying solvent is alpha when the time horizon is short and the bankroll is finite.

## 6. Failure Modes and Mitigations

| LLM hallucinates a market that doesn't exist | pythia-delphi-adapter validates every market_id against the live Delphi feed before passing downstream.                 |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| All analysts agree but are all wrong         | pythia-forged backtest harness tracks per-analyst Brier scores; underperformers get auto-demoted via consensus weights. |
| Single bad trade blows the bankroll          | pythia-risk enforces quarter-Kelly + hard per-market cap + portfolio exposure cap + circuit breaker on drawdown.        |
| ATT API goes down mid-trade                  | Executor catches 5xx, writes a TradeReceipt with status=PENDING, retries with idempotency key.                          |

| Signing key leaks                 | All signing keys are env-only, never persisted to disk; audit log records the fingerprint, not the key.         |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Reviewer can't understand a trade | Replay UI reconstructs the full decision chain on one screen — market → estimates → consensus → risk → receipt. |

## 7. Distribution

Pythia is distributed as a single flat monorepo, pushed to GitHub under the icohangar-ops organization. The repository includes the full source, installable packages, the technical design doc, and an architecture diagram.

| Code repository      | <a href="https://github.com/icohangar-ops/pythia">github.com/icohangar-ops/pythia</a> | Full source, installable, tests passing.   |
|----------------------|---------------------------------------------------------------------------------------|--------------------------------------------|
| Technical design doc | <a href="#">docs/pythia-design.pdf</a> (this document)                                | Architecture, rationale, math, roadmap.    |
| Architecture diagram | <a href="#">docs/assets/architecture.png</a>                                          | Layered pipeline visualization.            |
| Live dashboard       | <a href="#">packages/observability replay UI</a>                                      | Operators can replay any trade end-to-end. |

### 7.1 Repository layout

Pythia is a flat monorepo. Eight Python packages live under `packages/`, each independently pip-installable but designed to work together. The top-level `Makefile` installs them in dependency order via `make install`.

### 7.2 Component status

| <code>packages/delphi-adapter</code> | SDK bridge client + market models + settlement listener · 26 tests passing |
|--------------------------------------|----------------------------------------------------------------------------|
| <code>packages/analyst-mesh</code>   | 4 specialist analysts + registry + runner · 41 tests passing               |
| <code>packages/consensus</code>      | Fusion + agreement gate + audit signer · 39 tests passing                  |
| <code>packages/risk</code>           | Multi-outcome Kelly sizing + 6-gate pipeline · 27 tests passing            |
| <code>packages/executor</code>       | Full pipeline + CLI + paper/live modes · 16 tests passing                  |
| <code>packages/observability</code>  | Audit reader + FastAPI replay UI + achievements · 88 tests passing         |
| <code>packages/forge</code>          | Backtester + report generator + tune CLI · 31 tests passing                |
| <code>packages/strata</code>         | Enricher + 3 provider stubs · 60 tests passing                             |

Total: 8 modules, 328 tests passing across the reference implementation. The upstream icohangar-ops repos (`consensus-hardening-protocol`, `meshcfo`, `metabocommand`, `agent-observability`, `achievements`, `forge`, `strata`) remain the source of truth for the wrapped concerns and can be wired under `packages/*/vendor/` as vendored code or git submodules.